PRÁCTICA PASO A PASO

Guía Práctica: Desarrollo de Componentes Interactivos con React

Desarrollo de Software

Ruta de aprendizaje

• Configurar un entorno de desarrollo moderno para aplicaciones React utilizando herramientas actuales.

• Implementar componentes funcionales reutilizables aplicando el paso de propiedades (props) y la gestión de estados locales (useState).

• Construir una aplicación interactiva completa conectando eventos de usuario y efectos secundarios (useEffect).

ING. SISTEMAS
Duración: 90 minutos
Diagnóstico sintético

APARTADO 01

1. Introducción y Fundamentos de React

1. Introducción y Fundamentos de React

El desarrollo moderno de interfaces de usuario ha evolucionado desde el DOM imperativo tradicional hacia paradigmas declarativos impulsados por librerías como React. En el enfoque imperativo, el desarrollador detalla paso a paso cada modificación que debe sufrir la interfaz ante un evento. En contraste, el paradigma declarativo de React permite describir el estado visual de la aplicación en función de los datos actuales, delegando en el motor de la librería la tarea de actualizar eficientemente el DOM cuando dichos datos cambian.

La arquitectura de React se sustenta firmemente en la modularidad y la reutilización a través de componentes. Un componente es una porción independiente y autocontenida de interfaz que encapsula su propia estructura visual, lógica de comportamiento y estilo. Esta arquitectura basada en componentes facilita enormemente la escalabilidad en proyectos de ingeniería de sistemas, permitiendo que equipos de desarrollo trabajen de forma paralela en distintas secciones de una aplicación compleja sin generar conflictos de integración.

El ecosistema de React se complementa con herramientas de compilación avanzadas, enrutadores y gestores de estado global, formando una base sólida para aplicaciones web robustas. A lo largo de esta guía, utilizaremos componentes funcionales modernos apoyados en Hooks, un mecanismo que permite incorporar lógica de estado y efectos secundarios sin necesidad de recurrir a clases tradicionales.

Comparativa de Paradigmas de Desarrollo

Criterio	Desarrollo Imperativo (DOM Tradicional)	Desarrollo Declarativo (React)
Enfoque principal	Cómo cambiar la interfaz paso a paso.	Qué mostrar según el estado actual.
Manipulación del DOM	Manual, directa y propensa a errores.	Automatizada mediante el DOM virtual.
Escalabilidad	Baja en aplicaciones grandes y complejas.	Alta gracias a la arquitectura de componentes.
Mantenimiento	Complejo debido al acoplamiento estrecho.	Modular y simplificado por la separación de responsabilidades.

APARTADO 02

2. Requisitos Previos y Entorno de Desarrollo

2. Requisitos Previos y Entorno de Desarrollo

Para iniciar el desarrollo de aplicaciones modernas con React, es fundamental configurar un entorno de trabajo robusto. A diferencia de las configuraciones tradicionales basadas en Webpack, hoy en día se utiliza Vite como herramienta de construcción (bundler), ya que ofrece un rendimiento superior y un servidor de desarrollo extremadamente rápido gracias al uso de módulos nativos de ECMAScript en el navegador.

El primer paso consiste en verificar la instalación de Node.js y un gestor de paquetes como npm o pnpm en su sistema operativo. Node.js es indispensable porque provee el motor de ejecución para las herramientas de desarrollo y dependencias del proyecto.

BASH

```
Verificación de versiones en la terminal:
node -v
npm -v
```

Una vez confirmado el entorno base, procedemos a crear un nuevo proyecto ejecutando el asistente de configuración de Vite. Este comando descargará la plantilla base para React utilizando JavaScript o TypeScript según su preferencia.

BASH

```
Creación e inicialización del proyecto:
# 1. Crear el proyecto con Vite
npm create vite@latest mi-app-react -- --template react

# 2. Entrar al directorio
cd mi-app-react

# 3. Instalar dependencias
npm install

# 4. Iniciar servidor de desarrollo
npm run dev
```

APARTADO 03

3. Estructura del Proyecto y Sintaxis JSX

3. Estructura del Proyecto y Sintaxis JSX

Tras inicializar la aplicación con Vite, la estructura del proyecto se organiza de forma modular en el directorio raíz. La carpeta src es el núcleo donde reside la lógica de desarrollo, destacando el archivo main.jsx como el punto de entrada que monta el componente raíz App.jsx en el DOM del navegador. Para construir las interfaces, React emplea JSX, una extensión de sintaxis para JavaScript que permite describir la UI de manera declarativa combinando marcado y lógica. Entre las reglas fundamentales de JSX destacan la obligatoriedad de devolver un único elemento contenedor raíz —pudiendo utilizar fragmentos vacíos <>...</> si no se requiere un nodo HTML adicional—, la sustitución del atributo tradicional class por className para la asignación de clases CSS, y el uso de llaves {} para evaluar e interpolar expresiones de JavaScript directamente dentro del marcado de forma segura.

```jsx
import React from 'react';

export function TarjetaCurso({ titulo }) {
  const anio = 2024;
  return (
    <div className="tarjeta">
      <h2>{titulo}</h2>
      <span>Año: {anio}</span>
    </div>
  );
}
```

El bloque de código anterior demuestra la aplicación práctica de estas normas en un componente funcional moderno. La función TarjetaCurso recibe una propiedad y utiliza la sintaxis de className para aplicar estilos en el contenedor div, mientras que las llaves {} permiten inyectar dinámicamente el valor de la variable anio y la propiedad titulo directamente en el DOM virtual.

APARTADO 04

4. Creación de Componentes y Uso de Props

Creación de Componentes y Uso de Props en React

La arquitectura de React se fundamenta en la modularización de la interfaz de usuario mediante componentes funcionales. Un componente es una función de JavaScript que acepta entradas denominadas props (propiedades) y retorna elementos JSX que describen la interfaz visual. El uso de props permite que los componentes sean reutilizables y dinámicos, ya que actúan como parámetros de configuración que fluyen de manera unidireccional desde los componentes padres hacia los componentes hijos. Para mantener un código limpio y mantenible en Ingeniería de Sistemas, es indispensable estructurar cada componente en archivos independientes dentro del directorio src/components, aplicando la desestructuración de objetos en los parámetros para identificar claramente los datos esperados y evitar errores de tipado o referencias nulas en tiempo de ejecución.

Ejemplo de Componente Reutilizable con Props

```jsx
export function TarjetaUsuario({ nombre, rol, activo }) {
  return (
    <div className="tarjeta">
      <h3>{nombre}</h3>
      <p>Rol: {rol}</p>
      <span className={activo ? "badge-activo" : "badge-inactivo"}>
        {activo ? "En línea" : "Desconectado"}
      </span>
    </div>
  );
}
```

El código anterior define un componente funcional llamado TarjetaUsuario que recibe tres props: nombre, rol y activo. La desestructuración se realiza directamente en la firma de la función. En el JSX se aplican expresiones condicionales mediante el operador ternario para modificar dinámicamente las clases CSS y el texto mostrado según el estado del usuario. El resultado esperado es un bloque visual independiente que puede instanciarse múltiples veces en la aplicación pasando diferentes valores en sus atributos.

1. Componente Padre (App) define los datos iniciales
2. Se invocan los componentes hijos pasando atributos (props)
3. El componente hijo recibe las props como parámetros de función
4. El componente renderiza el JSX adaptado a los datos recibidos

APARTADO 05

5. Gestión de Estado Local con useState

5. Gestión de Estado Local con useState

A diferencia de las props, que son inmutables y fluyen de manera descendente desde el componente padre hacia el hijo, el estado local representa información interna que puede cambiar a lo largo del tiempo debido a interacciones del usuario, respuestas de red o temporizadores. En React, el estado se gestiona mediante el Hook useState, una función especial que permite a los componentes funcionales recordar información entre renderizados sucesivos.

Cuando se invoca a `useState`, este retorna un arreglo con dos elementos: el valor actual del estado y una función actualizadora. Cada vez que se ejecuta la función actualizadora, React programa una actualización del componente, recalcula la interfaz de usuario de manera eficiente utilizando el DOM virtual y refleja los cambios visuales correspondientes de forma inmediata y sincronizada.

JAVASCRIPT

```
import React, { useState } from 'react';

export function Contador() {
  const [contador, setContador] = useState(0);

  function incrementar() {
    setContador(contador + 1);
  }

  return (
    <div>
      <p>Valor actual: {contador}</p>
      <button onClick={incrementar}>Sumar</button>
    </div>
  );
}
```

El código anterior declara un estado llamado `contador` inicializado en cero. La función `incrementar` invoca a `setContador` sumando uno al valor previo. Al hacer clic en el botón, el componente se vuelve a renderizar automáticamente mostrando el nuevo número en pantalla sin recargar la página.

APARTADO 06

6. Ejemplo Resuelto: Aplicación de Tareas (Todo List)

6. Ejemplo Resuelto: Aplicación de Tareas (Todo List)

Para consolidar los conceptos de componentes y gestión de estado mediante el Hook useState, desarrollaremos una aplicación interactiva de tareas. Este ejercicio práctico permite comprender cómo fluyen los datos de manera unidireccional y cómo el cambio en el estado local actualiza la interfaz en tiempo real. La arquitectura emplea un campo de entrada controlado y una lista dinámica que reacciona a las acciones del usuario mediante eventos de formulario.

JAVASCRIPT

```
import React, { useState } from 'react';
export default function TodoApp() {
  const [tasks, setTasks] = useState([]);
  const [input, setInput] = useState('');
  const addTask = (e) => {
    e.preventDefault();
    if (!input.trim()) return;
    setTasks([...tasks, { id: Date.now(), text: input }]);
    setInput('');
  };
  return (
    <div>
      <form onSubmit={addTask}>
        <input value={input} onChange={(e) => setInput(e.target.value)} />
        <button type="submit">Agregar</button>
      </form>
      <ul>{tasks.map(t => <li key={t.id}>{t.text}</li>)}</ul>
    </div>
  );
}
```

1. El usuario escribe en el input actualizando el estado de texto.
2. Se presiona el botón para disparar el evento de envío.
3. Se genera un nuevo arreglo combinando el estado anterior con la tarea.
4. React renderiza nuevamente la lista actualizada en la interfaz.

APARTADO 07

7. Manejo de Efectos Secundarios con useEffect

7. Manejo de Efectos Secundarios con useEffect

En React, los efectos secundarios abarcan operaciones como solicitudes a APIs o temporizadores que escapan al renderizado puro. El Hook useEffect permite sincronizar componentes con sistemas externos ejecutando código después de que el navegador pinta la interfaz. Su comportamiento se rige por el arreglo de dependencias: si está vacío `[]`, se ejecuta solo al montar el componente; si contiene variables, se dispara cada vez que alguna de ellas cambia. Esto evita bloqueos y gestiona correctamente el ciclo de vida.

```jsx
import { useState, useEffect } from 'react';

export function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`https://api.example.com/users/${userId}`)
      .then(res => res.json())
      .then(data => setUser(data));
  }, [userId]);

  return <div>{user ? user.name : 'Cargando...'}</div>;
}
```

En el ejemplo, `useEffect` ejecuta la petición `fetch` cada vez que el valor de `userId` cambia en el arreglo de dependencias. La función de limpieza opcional evita fugas de memoria al desmontar componentes.

APARTADO 08

8. Errores Frecuentes y Buenas Prácticas

8. Errores Frecuentes y Buenas Prácticas

Durante el desarrollo de aplicaciones web utilizando React, es común que los estudiantes y desarrolladores cometan errores relacionados con la inmutabilidad del estado y el manejo del ciclo de vida de los componentes. Uno de los fallos más frecuentes consiste en modificar directamente la variable de estado en lugar de utilizar la función de actualización provista por el Hook useState. Esta práctica inadecuada impide que el motor de React detecte los cambios, lo que ocasiona que la interfaz de usuario no se vuelva a renderizar de forma sincronizada con los datos internos.

Otro error crítico ocurre al omitir las dependencias requeridas dentro del Hook useEffect, lo que frecuentemente desencadena bucles infinitos de peticiones de red o ejecuciones descontroladas de código. Para mantener un código limpio y mantenible, se recomienda estructurar los componentes de manera modular, separando la lógica de negocio de la presentación visual y aplicando convenciones de nomenclatura claras. A continuación, se muestra un ejemplo que contrasta la forma incorrecta de actualizar un arreglo en el estado frente a la forma correcta e inmutable.

JAVASCRIPT

```
import { useState } from 'react';

export function ListaInvalida() {
  const [items, setItems] = useState(['Elemento 1']);

  const agregarItemIncorrecto = () => {
    items.push('Elemento 2'); // ERROR: Modificación directa
    setItems(items);
  };

  const agregarItemCorrecto = () => {
    setItems([...items, 'Elemento 2']); // CORRECTO: Inmutabilidad
  };

  return <div>Resultado</div>;
}
```

El código anterior ilustra la importancia de utilizar el operador de propagación (...) para generar una nueva referencia de memoria en el arreglo. El resultado esperado de aplicar esta buena práctica es que el componente detecte el cambio de manera predecible, optimizando el rendimiento general de la aplicación y evitando comportamientos erráticos en la interfaz.

Error Común	Impacto Técnico	Solución Recomendada
Modificar el estado directamente	La interfaz no se actualiza	Usar siempre la función setter
Mutar arreglos con push() o pop()	Pérdida de inmutabilidad	Emplear métodos como map, filter o spread
Faltan dependencias en useEffect	Bucles infinitos o datos obsoletos	Incluir todas las variables externas usadas

APARTADO 09

9. Ejercicios Propuestos y Criterios de Evaluación

9. Ejercicios Propuestos y Criterios de Evaluación

Para consolidar los conocimientos adquiridos a lo largo de esta guía práctica de desarrollo en React, se plantean dos ejercicios autónomos orientados a evaluar la capacidad de diseño modular, gestión de estados complejos y consumo de datos asíncronos. Los estudiantes deben aplicar los fundamentos de componentes funcionales, props, los Hooks useState y useEffect, así como las buenas prácticas de estructuración de código limpio abordadas en las secciones anteriores. Estos ejercicios simulan escenarios reales del desarrollo de software moderno y permiten verificar el dominio técnico antes de la calificación final.

Enunciado de los Ejercicios Prácticos

El primer ejercicio consiste en desarrollar un Catálogo de Productos con Filtro en Tiempo Real, el cual debe incluir un campo de búsqueda interactivo y un sistema de categorías que actualice la interfaz de manera instantánea utilizando el Hook useState. El segundo ejercicio requiere la implementación de un Tablero del Clima que consuma datos mediante fetch dentro de un Hook useEffect, mostrando la temperatura actual, humedad y manejo de estados de carga y error para garantizar una experiencia de usuario robusta y libre de fallos en tiempo de ejecución.

Rúbrica de Evaluación del Desempeño

Criterio	Nivel Excelente (4 pts)	Nivel Satisfactorio (3 pts)	Nivel Deficiente (1-2 pts)
Funcionalidad	La aplicación cumple con todos los requisitos y opera sin errores.	La aplicación opera pero presenta fallos menores en algún evento.	La aplicación no compila o carece de funciones clave.
Código Limpio	Usa componentes modulares, nombres descriptivos y buenas prácticas.	Estructura aceptable pero con componentes poco reutilizables.	Código monolítico, desorganizado y sin convenciones.
Manejo de Estado	Uso correcto de useState y useEffect sin fugas de memoria.	Uso básico de estado pero con re-renderizados innecesarios.	Ausencia de manejo de estado reactivo.